

Digital Egypt Pioneers Initiative (DEPI)

Graduation Project — Deliverable File 4

SYSTEM ANALYSIS & DESIGN

Houra Platform

Use Case, ERD, DFD, Sequence, Activity, State & Class Diagrams | UI/UX | Architecture

Prepared By:

#	Team Member Name
1	Amany Mohamed Ebrahim Belal
2	Nourhan Taha Abdelmonem Mohamed
3	Ahmed Alaa Abdelsalam Ali
4	Hussein Rezk Abdelrazak Elgorany
5	Nourhan Moustafa Kamel Mohamed

Date: 2026

Table of Contents

- Table of Contents 2
- 1.1 Problem Statement (Recap) 3
- 1.2 Project Objectives (Recap) 3
- 1.3 Use Case Diagram & Descriptions 3
- 1.4 Functional & Non-Functional Requirements (Summary)..... 4
- 1.5 Software Architecture 5
- 2.1 Entity-Relationship Diagram (ERD) 6
- 2.2 Logical & Physical Schema..... 7
- 3.1 Data Flow Diagram — Context Level (Level 0) 10
- 3.2 Data Flow Diagram — Level 1 10
- 3.3 Sequence Diagram — Requesting & Completing a Service Exchange 11
- 3.4 Sequence Diagram — Provider Verification..... 11
- 3.5 Activity Diagram — Service Exchange Workflow 13
- 3.6 State Diagram — Service Request Lifecycle 14
- 3.7 Class Diagram 15
- 4.1 Wireframes & Mockups 16
- 4.2 UI/UX Guidelines 24
- 5.1 Technology Stack 26
- 5.2 Component Diagram..... 26
- 5.3 Deployment Diagram 28
- 5.4 Deployment Strategy 28
- Conclusion..... 28

PART 1 — PROBLEM STATEMENT, OBJECTIVES & ARCHITECTURE

1.1 Problem Statement (Recap)

Many individuals possess valuable skills and free time but lack the financial resources to access paid services or training, while many people who are willing to share their expertise have no structured, trust-based platform through which to do so. The Time Banking Platform addresses this by enabling members to exchange skills and services using time — rather than money — as the unit of value.

1.2 Project Objectives (Recap)

- Design and deliver a functional platform that lets users exchange skills and services fairly using time credits.
- Provide a reliable, secure, and trustworthy environment through profile verification and a rating system.
- Support the full exchange lifecycle: registration, listing, requesting, completing, and rating a service.

1.3 Use Case Diagram & Descriptions

The diagram below identifies the three primary actors of the system — Service Seeker, Service Provider, and Administrator — and the main use cases each actor interacts with.

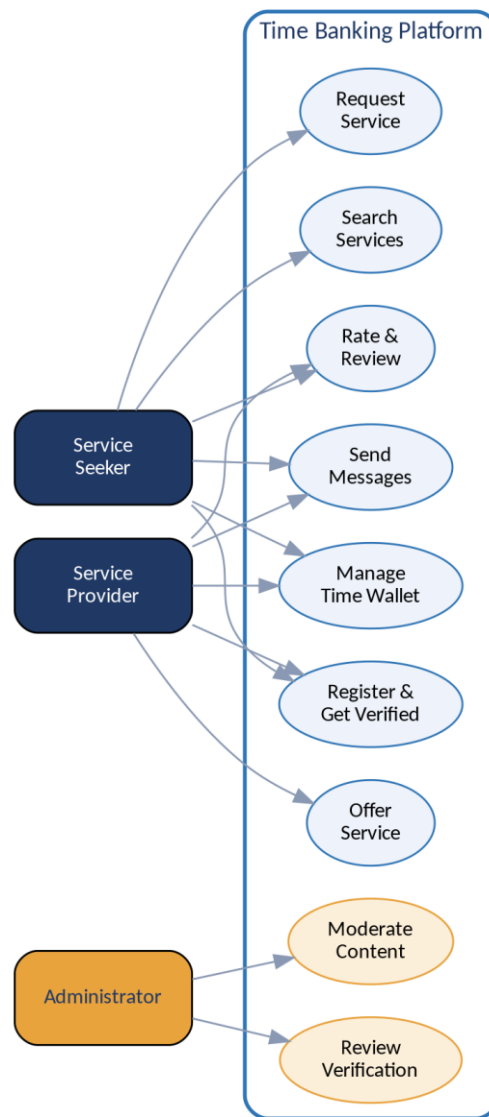


Figure 1.1 — Use Case Diagram: Houra Platform

Actor	Description	Key Use Cases
Service Seeker	A user looking to receive a skill/service in exchange for time credits.	Register & Get Verified, Search Services, Request Service, Manage Time Wallet, Rate & Review, Send Messages
Service Provider	A user offering a skill/service in exchange for earning time credits.	Register & Get Verified, Offer Service, Manage Time Wallet, Rate & Review, Send Messages
Administrator	Platform staff responsible for trust, safety, and content moderation.	Review Verification, Moderate Content

1.4 Functional & Non-Functional Requirements (Summary)

Full requirement specifications (FR-01 through FR-14, and all non-functional requirements) were defined in the Requirements Gathering file. The table below summarizes the requirement categories for quick reference during design.

Category	Summary
Account & Verification	Registration, profile creation, certificate/video upload, admin approval workflow.
Time Wallet & Transactions	Starting balance, automatic credit/debit on exchange completion, demand-based weighting.
Service Discovery & Matching	Search, filter, request/accept workflow.
Trust & Community	Ratings, reviews, notifications, in-app messaging.
Non-Functional	Performance ($\leq 2s$ response), security (encryption), 99% uptime, usability, scalability.

1.5 Software Architecture

The system follows a Layered (N-Tier) Architecture, which cleanly separates concerns and aligns with the ASP.NET Core technology stack. This style was chosen over microservices for this project's scope, since it is simpler to build, test, and deploy for a single-team graduation project while still remaining scalable if the platform grows.

Layer	Technology	Responsibility
Presentation Layer	HTML5, CSS3, JavaScript rendered via ASP.NET Core MVC (Razor Views)	Renders the UI and handles user interaction in the browser.
API Layer	ASP.NET Core Web API (C#)	Exposes RESTful endpoints consumed by the presentation layer and, potentially, future mobile clients.
Business Logic Layer	C# Services (Exchange, Wallet, Verification, Rating)	Implements core rules: time-credit calculations, verification workflow, matching logic.
Data Access Layer	Entity Framework Core (ORM)	Maps C# objects to database tables and manages queries/transactions.
Database Layer	Microsoft SQL Server	Persists all platform data (users, listings, exchanges, ratings, etc.).

A full Component Diagram and Deployment Diagram illustrating how these layers map to running infrastructure are provided in Part 5 — System Deployment & Integration.

PART 2 — DATABASE DESIGN & DATA MODELING

2.1 Entity-Relationship Diagram (ERD)

The ERD below models the core entities of the Houira Platform and the relationships between them, including users, skills, service listings, exchanges, ratings, verification documents, notifications, and messages.

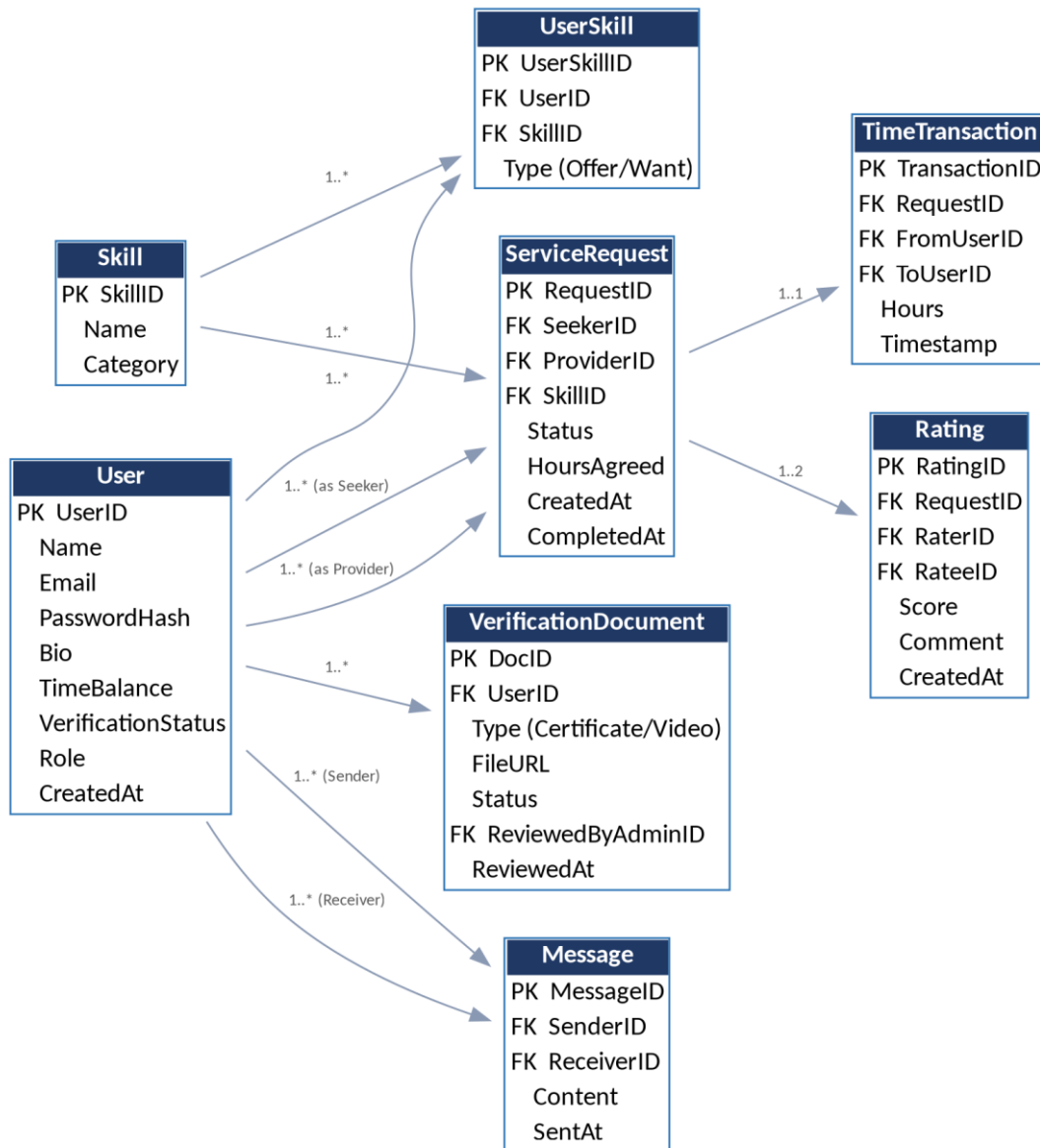


Figure 2.1 — Entity-Relationship Diagram

2.2 Logical & Physical Schema

The tables below define the attributes, data types, and keys for each entity, along with normalization notes (the schema is normalized to Third Normal Form — 3NF — to avoid redundancy).

USER

Attribute	Type	Key	Notes
UserID	INT	PK	Auto-increment, uniquely identifies each user.
Name	NVARCHAR(100)	—	Full name.
Email	NVARCHAR(150)	Unique	Used for login and notifications.
PasswordHash	NVARCHAR(255)	—	Securely hashed, never stored in plain text.
Bio	NVARCHAR(500)	—	Short self-introduction.
TimeBalance	DECIMAL(6,2)	—	Current time-credit balance, in hours.
VerificationStatus	NVARCHAR(20)	—	Pending / Approved / Rejected.
JoinDate	DATETIME	—	Account creation timestamp.

SKILL

Attribute	Type	Key	Notes
SkillID	INT	PK	Auto-increment.
Name	NVARCHAR(100)	—	e.g., “Graphic Design”, “French Language”.
Category	NVARCHAR(50)	—	e.g., Technology, Education, Arts.

USER_SKILL

Attribute	Type	Key	Notes
UserSkillID	INT	PK	Auto-increment.
UserID	INT	FK → USER	The associated user.
SkillID	INT	FK → SKILL	The associated skill.
Type	NVARCHAR(10)	—	‘Offered’ or ‘Wanted’.

SERVICE_LISTING

Attribute	Type	Key	Notes
ListingID	INT	PK	Auto-increment.
UserID	INT	FK → USER	The provider offering the listing.
SkillID	INT	FK → SKILL	The skill being offered.

Attribute	Type	Key	Notes
Description	NVARCHAR(1000)	—	Details of the offered service.
CreditCost	DECIMAL(4,2)	—	Hours required to redeem this service.
Status	NVARCHAR(20)	—	Active / Paused / Closed.

EXCHANGE

Attribute	Type	Key	Notes
ExchangeID	INT	PK	Auto-increment.
RequesterID	INT	FK → USER	The seeker requesting the service.
ProviderID	INT	FK → USER	The provider fulfilling the request.
ListingID	INT	FK → SERVICE_LISTING	The listing being exchanged.
HoursSpent	DECIMAL(4,2)	—	Actual time credited/debited.
Status	NVARCHAR(20)	—	Pending / Accepted / InProgress / Completed / Cancelled.
RequestDate	DATETIME	—	When the request was made.
CompletionDate	DATETIME	Nullable	When the exchange was completed.

RATING

Attribute	Type	Key	Notes
RatingID	INT	PK	Auto-increment.
ExchangeID	INT	FK → EXCHANGE	The exchange being rated.
RaterID	INT	FK → USER	The user giving the rating.
RatedUserID	INT	FK → USER	The user being rated.
Score	INT	—	1–5 star rating.
Comment	NVARCHAR(500)	Nullable	Optional written review.

VERIFICATION_DOCUMENT

Attribute	Type	Key	Notes
DocID	INT	PK	Auto-increment.
UserID	INT	FK → USER	The user submitting the document.
Type	NVARCHAR(20)	—	'Certificate' or 'Video'.

Attribute	Type	Key	Notes
FileURL	NVARCHAR(255)	—	Path/URL to the stored file.
Status	NVARCHAR(20)	—	Pending / Approved / Rejected.

NOTIFICATION

Attribute	Type	Key	Notes
NotificationID	INT	PK	Auto-increment.
UserID	INT	FK → USER	The recipient.
Message	NVARCHAR(255)	—	Notification text.
IsRead	BIT	—	Read/unread flag.

MESSAGE

Attribute	Type	Key	Notes
MessageID	INT	PK	Auto-increment.
SenderID	INT	FK → USER	The message sender.
ReceiverID	INT	FK → USER	The message recipient.
Content	NVARCHAR(1000)	—	Message body.
Timestamp	DATETIME	—	When the message was sent.

PART 3 — DATA FLOW & SYSTEM BEHAVIOR

3.1 Data Flow Diagram — Context Level (Level 0)

The context diagram shows the Houra Platform as a single process interacting with its three external entities.

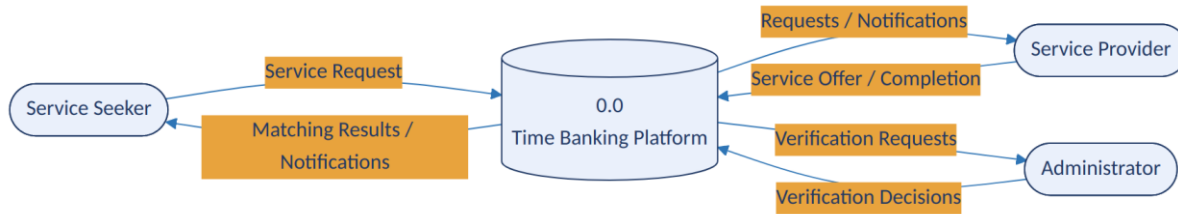


Figure 3.1 — DFD Level 0 (Context Diagram)

3.2 Data Flow Diagram — Level 1

The Level 1 diagram decomposes the system into four major processes and their associated data stores.

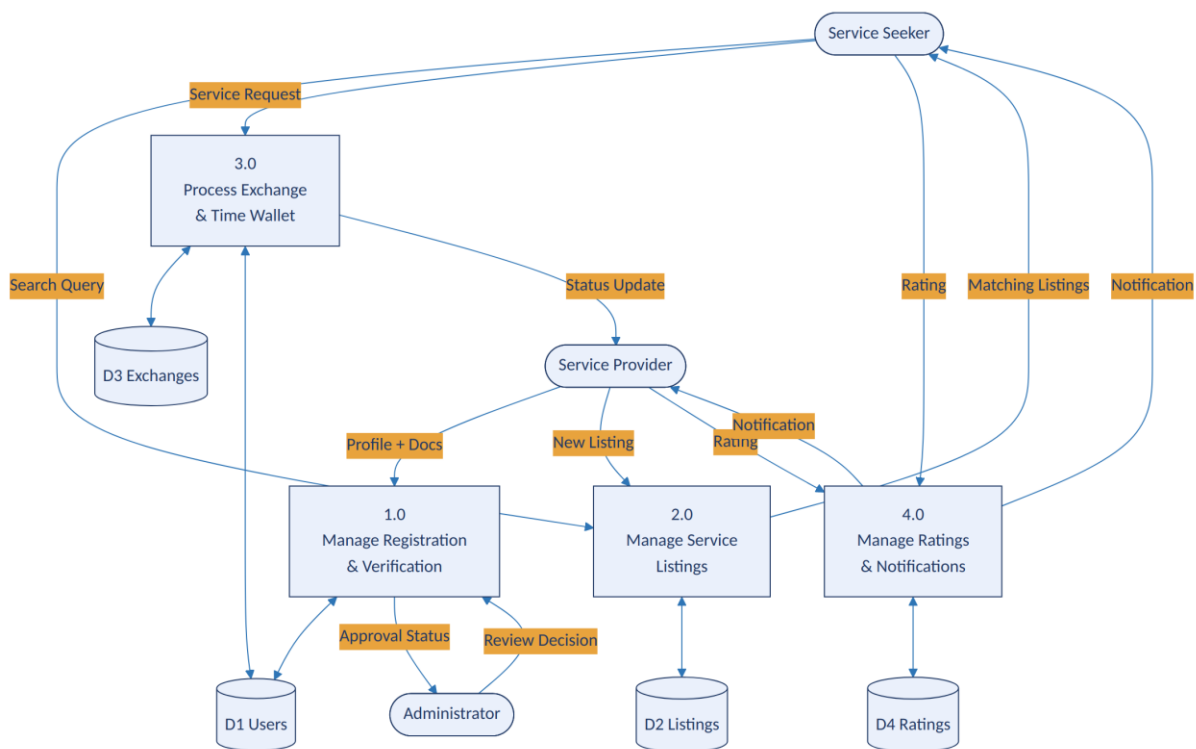


Figure 3.2 — DFD Level 1

3.3 Sequence Diagram — Requesting & Completing a Service Exchange

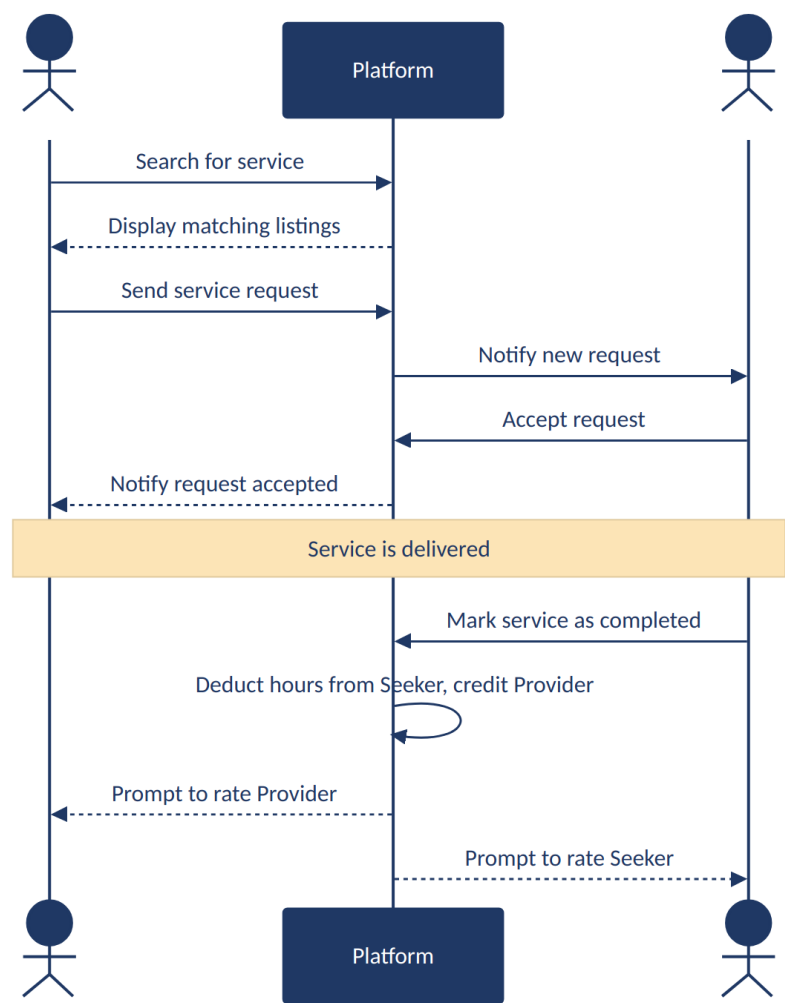


Figure 3.3 — Sequence Diagram: Service Exchange

3.4 Sequence Diagram — Provider Verification

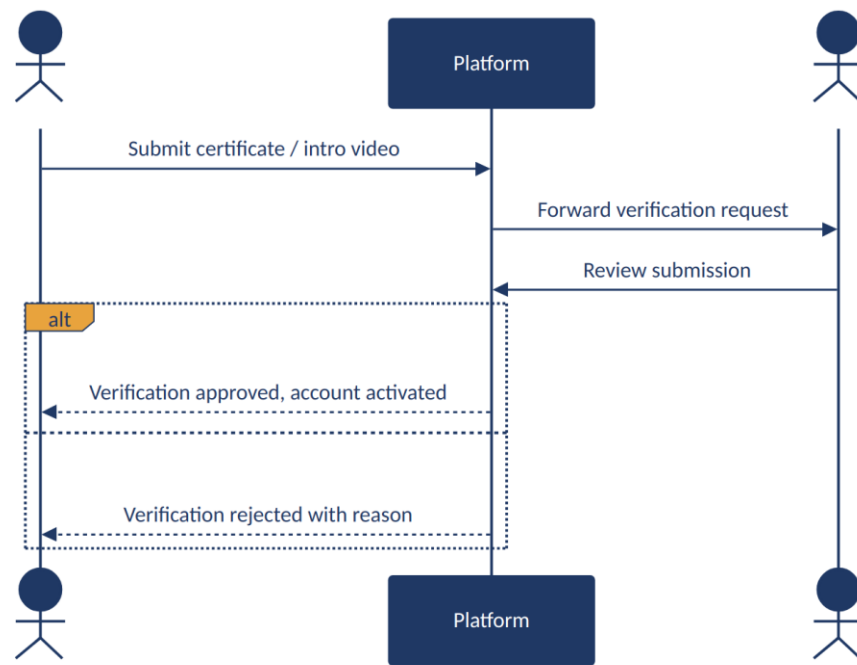


Figure 3.4 — Sequence Diagram: Provider Verification

3.5 Activity Diagram — Service Exchange Workflow

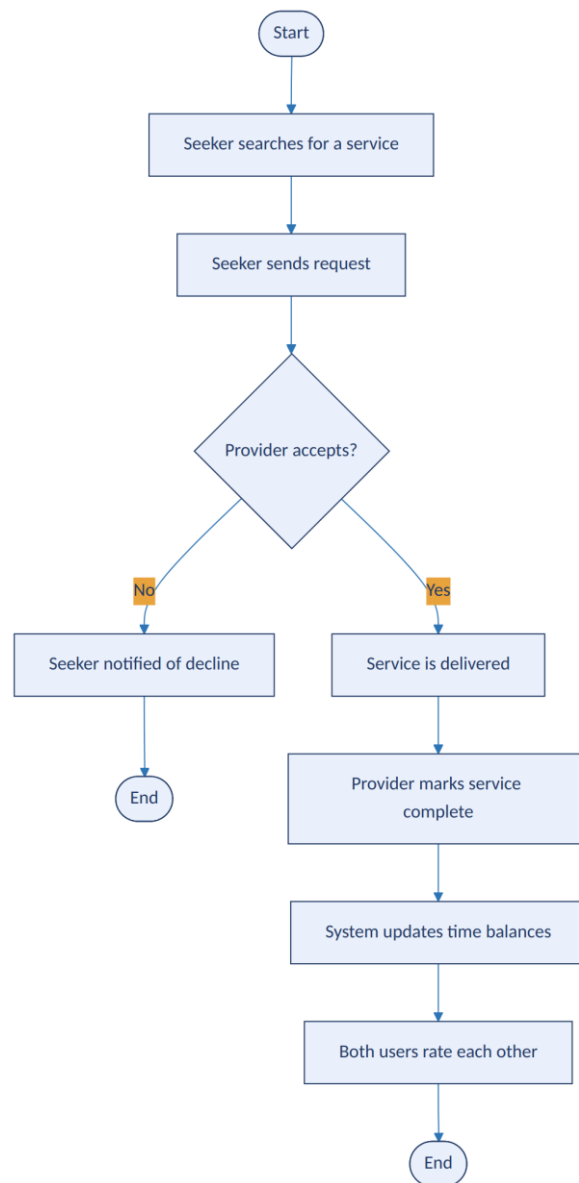


Figure 3.5 — Activity Diagram

3.6 State Diagram — Service Request Lifecycle

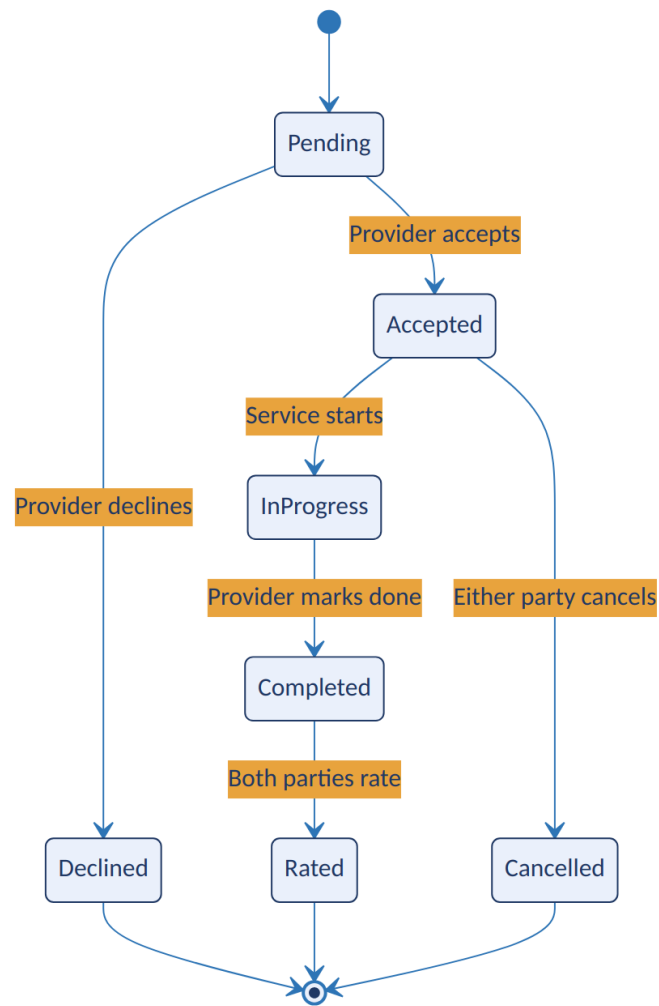


Figure 3.6 — State Diagram

3.7 Class Diagram

The class diagram below defines the core system classes, their attributes, methods, and relationships, including the inheritance relationship between Admin and User.

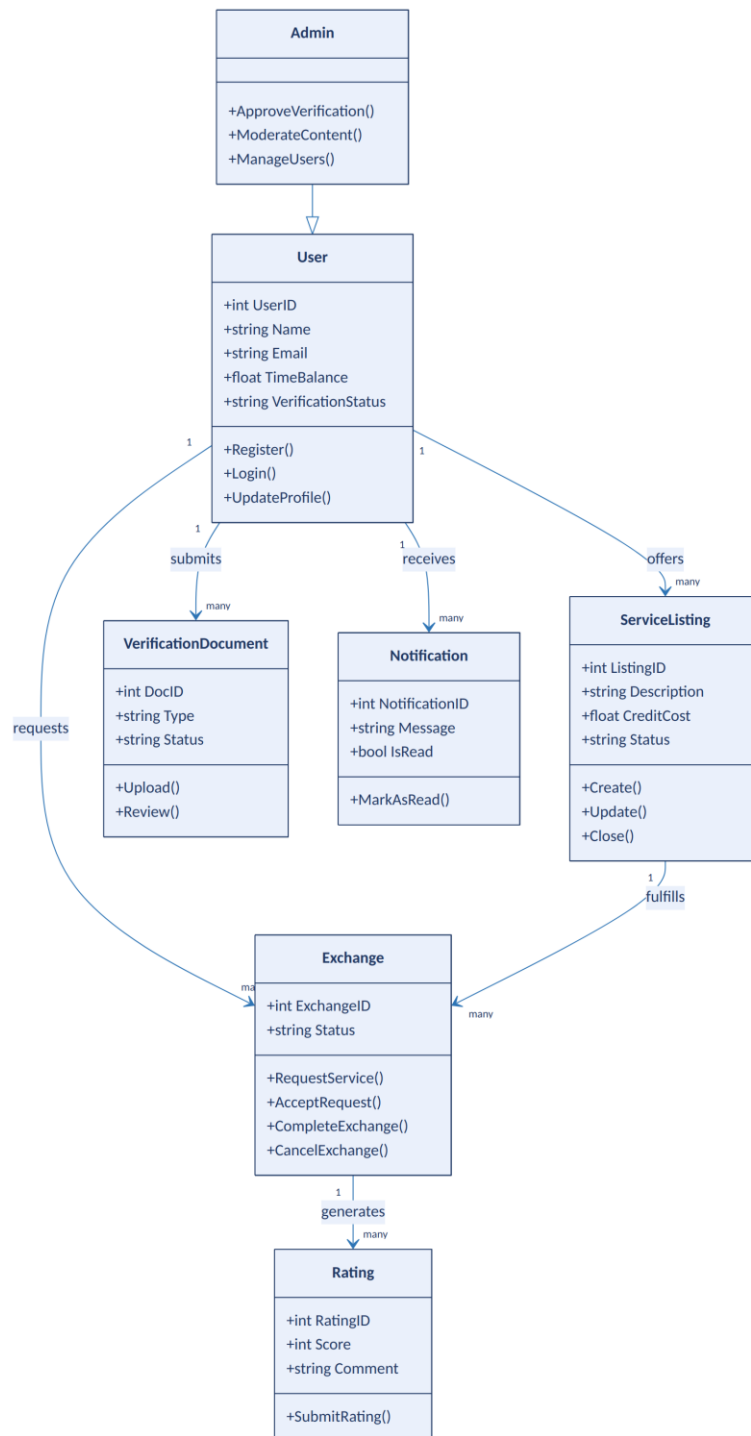
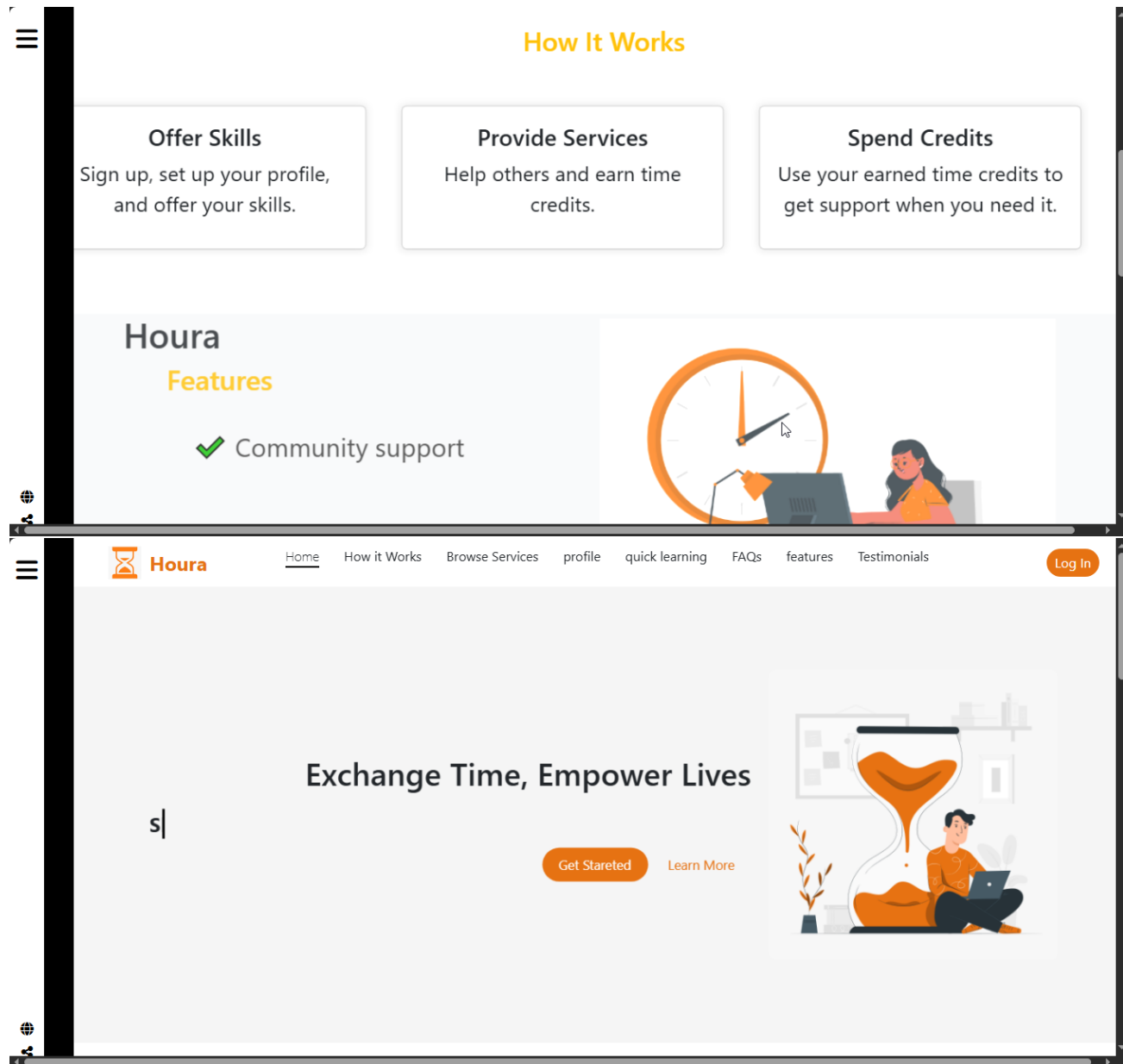


Figure 3.7 — Class Diagram

4.1 Wireframes & Mockups



Houra

Features

- ✓ Community support
- ✓ Secure Platform
- ✓ Easy to use
- ✓ Flexible exchanges



About Us

Organisation's mission is to foster a community by exchanging time and skills. At Allocate, each one assists the other.

Testimonials



Sarah



"A fantastic way to connect with others and share skills."



James



"I've earned and spent time credits easily. Love it!"



Maria



"The platform is user-friendly and very supportive."

Welcome Back to Houra

log in to exchange time & skills

Forget Password? [Don't have an account ?](#)



Sign Up

Already have an account? [Login](#)



PROFILE



skills :

Edit Profile

About Me

Service Offered

 **1 hour =**
1 hour of learning

Reviews

★★★★☆

"Share a little of your time, and gain a lot in return."

Timebanking Exchange



My Services

Search Servers

+ Create Server





You have no hours remaining

Buy Hours

Purchase additional hours to continue using Houra.

2 hours

\$5

Purchase

5 hours

\$10

Purchase

10 hours

\$20

Purchase



Houra

Quick Learning

A server for short lessons and simple educational sessions.

★★★★★ (4.7)

Courses

How to Master Time Managment

A brief guide to organizing your daily schedule effectively.

Instructor: Sara Ahmad · 2h

👤 850 participants

Requests

★★★★☆

4.5

Excel Basics in 1 Hour

Learn the most essential Excel functions fast and easy.

Instructor: Omar Khald · 1h

👤 1200 participants

Requests

★★★★★

5.0

home

my services

notification

activity

purchase hours

send notes

dashboard

login

sign up

browse services

profile

contact us



Copyright © 2019 All Rights Reserved.

Quick Learning

A server for short lessons and simple educational sessions.

★★★★★ (4.7)

× e Managment

schedule effectively.

Requests

★★★★☆

4.5

our

is fast and easy.

Requests

★★★★★

5.0



Home

my
services

Browse
Services

quick
learning

FAQs

my
profile

notification

go to
learn

purchase
hours

quick
learning

dashboard

Log In

We'd Love your feedback!

Help us improve Houra by rating experience.

★★★★★

What did you like most about Houra?

Your answer

Any suggestion to make it better?

Your suggestion

Submit Feedback

Frequently Asked Questions (FAQ)

Who can use the Houra platform? +

How does the Houra credit system work? +

Can I chat with someone before accepting an exchange? +

Is this only for local exchanges? +



Do you have more questions?

Want to know more about our premium service?
Meet the right platform to help you realize.

Submit

Dashboard

[Home](#) [MyServices](#) [Buy Hours](#) [Messages](#) [Notifications](#) [Requests](#) 



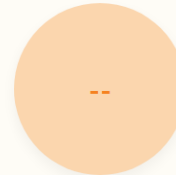
Active Services



Time Balance



Ongoing Requests



Completed Hours

contact us

Feel free to contact us at any time. We will get back to you as soon as we can.

Name

Email

Message

Send

Info

✉ info@gmail.com

☎ +03 123456

📞 +20 1273370343

📅 Sunday-Thursday

🔍 Search people

👤 Mary

👤 Mona

👤 John Doe



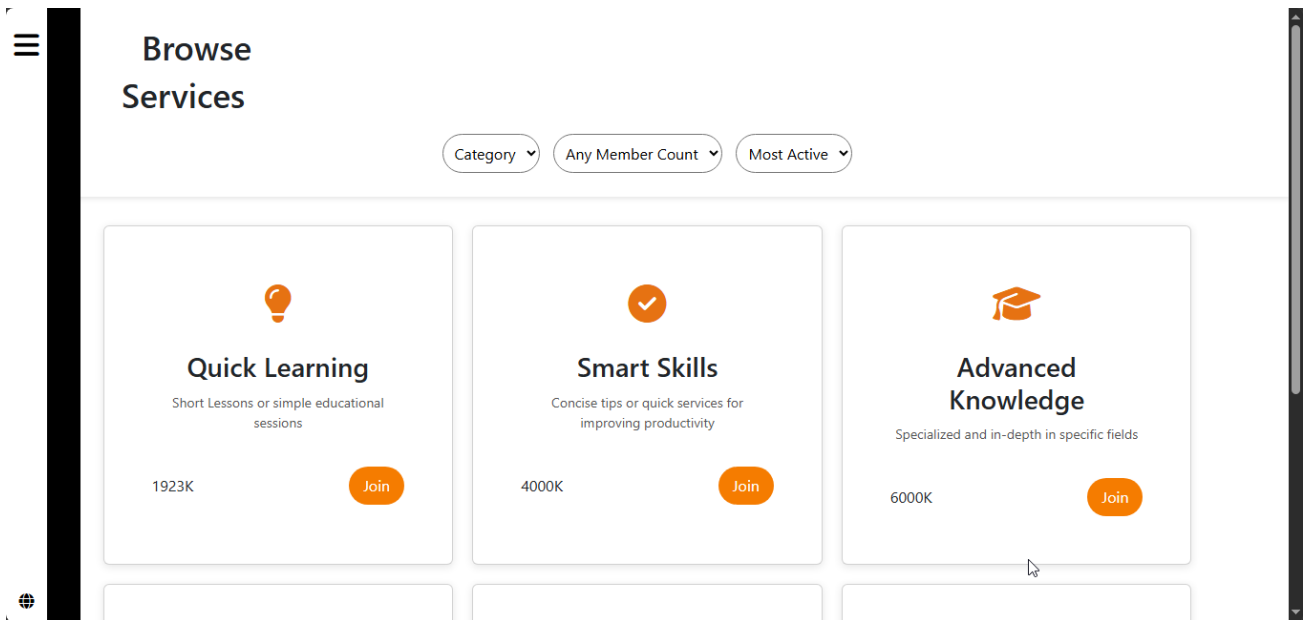
Note with Sarah

📌 Tomorrow's schedule confirmed at 4 PM.

📝 Remember to prepare the writing materials before the session.

Write a new note...

Add Note



4.2 UI/UX Guidelines

Color Palette

- Primary — Navy Blue (#1F3864): headers, primary buttons, key text.
- Accent — Amber/Gold (#E8A33D): highlights, icons, calls to attention (reflecting the “time/hourglass” theme).
- Supporting — Ice Blue (#EEF2FA) and white backgrounds for readability and a clean, trustworthy feel.

Typography

- Headings: a serif or semi-bold display font for a professional, trustworthy tone.
- Body text: Calibri or a similar clean sans-serif, minimum 14px on web for readability.

Accessibility

- Maintain a minimum contrast ratio of 4.5:1 between text and background (WCAG AA).
- All interactive elements must be keyboard-navigable and screen-reader friendly (proper alt text, ARIA labels).
- Form validation errors must be announced clearly, not conveyed by color alone.

Consistency Principles

- Reuse the same card, button, and badge components across all screens (profile cards, listing cards, rating badges).
- Use a consistent iconography set throughout (e.g., a single icon library) to reinforce recognition.
- Keep the verification/trust badge visually prominent wherever a user's profile appears.

5.1 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript (rendered via ASP.NET Core MVC Razor Views)
Backend / API	ASP.NET Core Web API (C#)
Web Application Framework	ASP.NET Core MVC
Database	Microsoft SQL Server
ORM	Entity Framework Core
Version Control	Git & GitHub
Containerization	Docker
Testing	Unit Testing (xUnit/MSTest) with functional documentation

5.2 Component Diagram

The component diagram shows the high-level software components of the system and their dependencies, consistent with the layered architecture defined in Part 1.

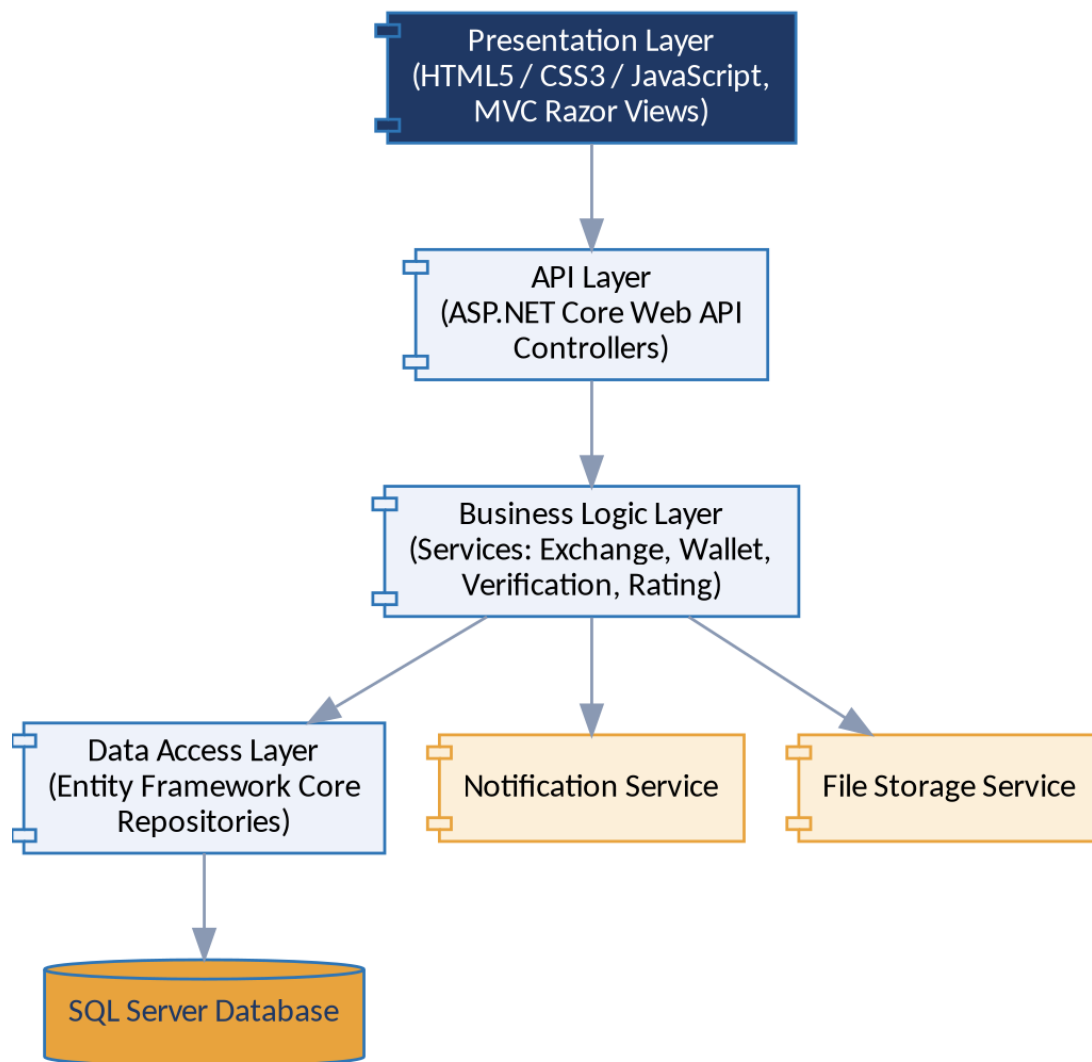


Figure 5.1 — Component Diagram

5.3 Deployment Diagram

The deployment diagram illustrates how the software components are distributed across physical/cloud infrastructure: a client browser, a containerized application server hosting the Web API and MVC application, a SQL Server database, and cloud storage for verification files.

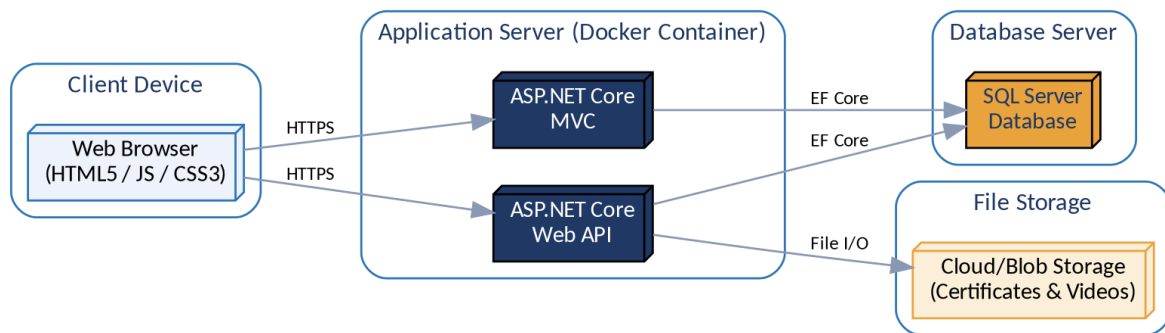


Figure 5.2 — Deployment Diagram

5.4 Deployment Strategy

- The application will be containerized using Docker to ensure consistent behavior across development and hosting environments.
- Source code will be version-controlled on GitHub, with a simple feature-branch workflow.
- The initial pilot will be deployed to a low-cost cloud tier (or university lab server), with the database backed up on a regular schedule.
- Detailed API documentation and the testing/validation plan will be produced alongside the Implementation and Testing & QA deliverable files.

Conclusion

This System Analysis & Design file translates the requirements defined earlier into a concrete technical blueprint: system actors and use cases, a normalized relational database, detailed process and behavioral diagrams, UI/UX guidelines, and a deployment-ready architecture built on the ASP.NET Core / SQL Server stack. Together with the team's own UI/UX wireframes, this file provides everything needed to begin implementation with confidence.